

iOS面试题合集（架构+设计模式篇）

iOSer iOSer 5月19日

收录于话题

#BAT大厂面试题合集

9个

欢迎点击上方蓝字“iOSer”关注我们
再点击右上角“...”菜单，选择“设为星标”
我们一起精进、成长！

本栏目将持续更新--请iOS的小伙伴关注我们!

做这个的初心是希望能巩固自己的基础知识，

当然也希望能帮助更多的开发者，

可以加入我们的技术交流群：**834688868**，群里每天都会有干货、技术动向、职业生涯、行业热点、职场趣事等一切有关于程序员的内容分享。

不管你是大牛还是小白都欢迎入驻，分享BAT，阿里面试题、面试经验，讨论技术，大家一起交流学习成长！

目录：

一，项目架构

- 1.MVC、MVP、MVVM模式
- 2.关于RAC你有怎样运用到解决不同API依赖关系
- 3.@weakify和我们宏定义的weakSelf有什么区别？
4. 微服务的架构设想

二，设计模式

1. iOS有哪些常见的设计模式？

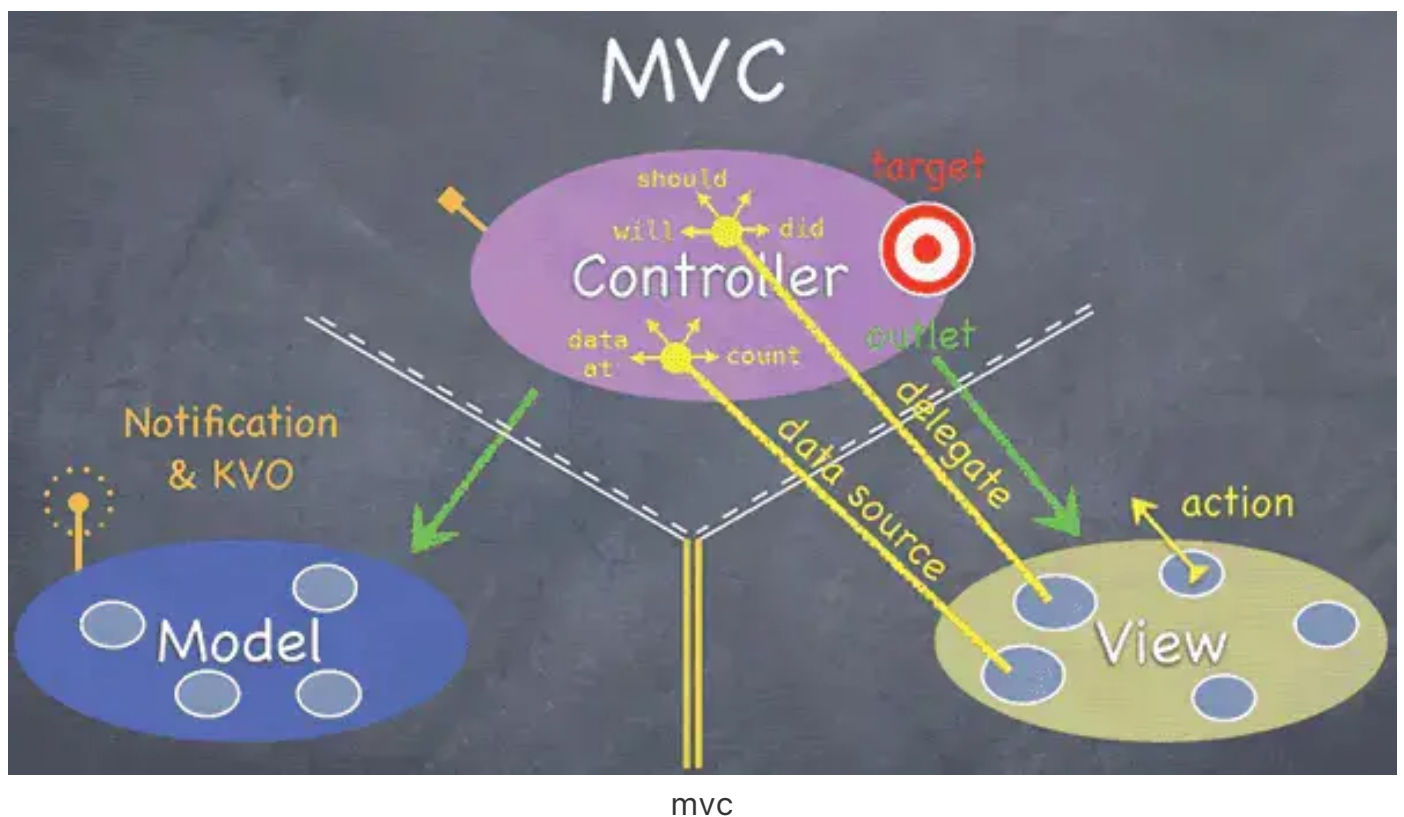
- 2.单例会有什么弊端？
3. 编程中的六大设计原则？
- 4.如何设计一个图片缓存框架？
- 5.如何设计一个时长统计框架？

一，项目架构篇

1.MVC、MVP、MVVM模式

MVC (Model、View、Controller)

MVC是比较直观的架构模式，最核心的就是通过Controller层来进行调控，首先看一下官方提供的MVC示意图：



- Model和View永远不能相互通信，只能通过Controller传递

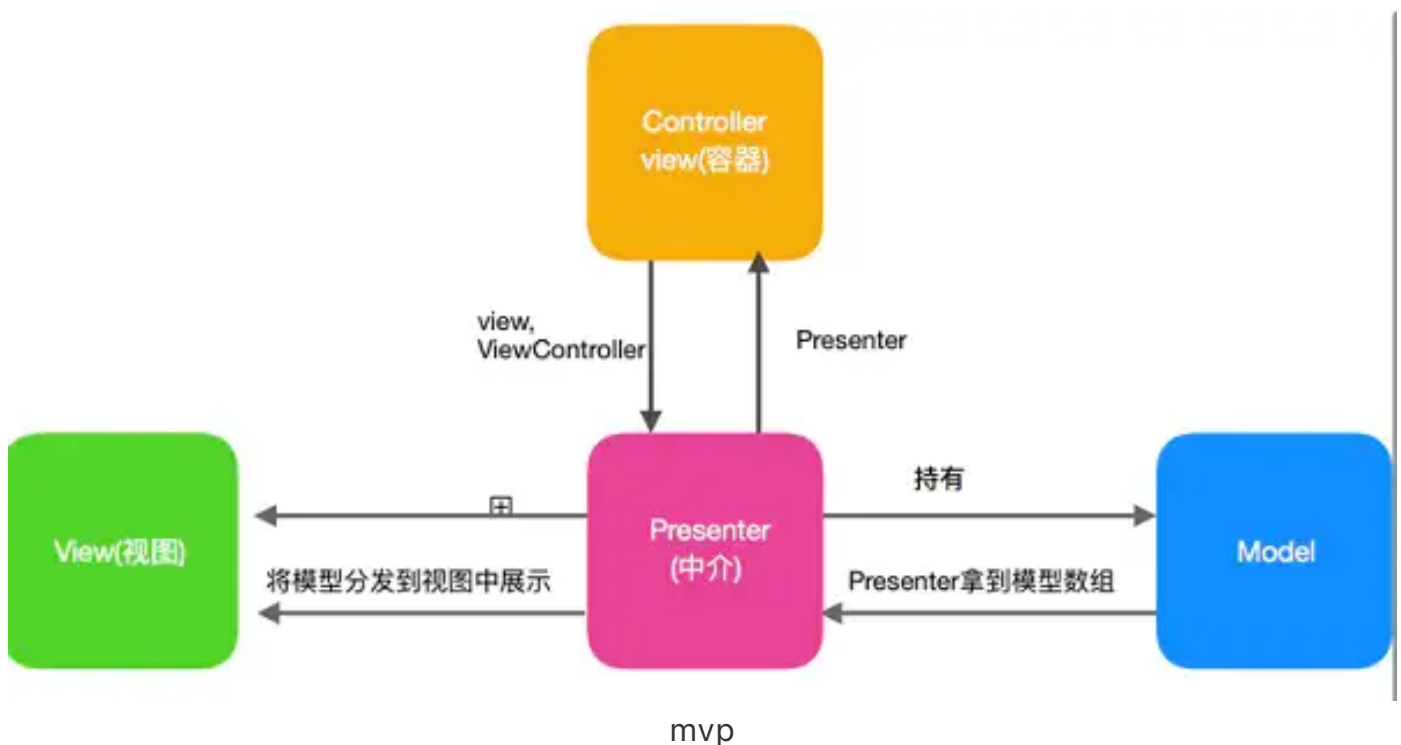
- Controller可以直接与Model对话（读写调用Model），Model通过NOtification和KVO机制与Controller间接通信

Controller可以直接与View对话，通过IBoutlet直接操作View，IBoutlet直接对应View的控件（例如创建一个Button：需声明一个 IBOutlet UIButton * btn），View通过action向Controller报告事件的发生(用户点击了按钮)。Controller是View的直接数据源

- 优点：对于混乱的项目组织方式，有了一个明确的组织方式。通过Controller来掌控全局，同时将View展示和Model的变化分开
- 缺点：愈发笨重的Controller，随着业务逻辑的增加，大量的代码放进Controller，导致Controller越来越臃肿，堆积成千上万行代码，后期维护起来费时费力

MVP (Model、View、Presenter)

MVP模式是MVC模式的一个演化版本，其中Model与MVC模式中Model层没有太大区别，主要提供数据存储功能，一般都是用来封装网络获取的json数据；View与MVC中的View层有一些差别，MVP中的View层可以是viewController、view等控件；Presenter层则是作为Model和View的中介，从Model层获取数据之后传给View。



从上图可以看出，从MVC模式中增加了Presenter层，将UIViewController中复杂的业务逻辑、网络请求等剥离出来。

- 优点 模型和视图完全分离，可以做到修改视图而不影响模型；更高效的使用模型，View不依赖Model，可以说View能做到对业务逻辑完全分离
- 缺点 Presenter中除了处理业务逻辑以外，还要处理View-Model两层的协调，也会导致Presenter层的臃肿

MVVM (Model、Controller/View、ViewModel)

在MVVM中，view和ViewControlller联系在一起，我们把它们视为一个组件，view和ViewControlller都不能直接引用model，而是引用是视图模型即ViewModel。viewModel是一个用来放置用户输入验证逻辑、视图显示逻辑、网络请求等业务逻辑的地方，这样的设计模式，会轻微增加代码量，但是会减少代码的复杂性

- 优点 View可以独立于Model的变化和修改，一个ViewModel可以绑定到不同的View上，降低耦合，增加重用
- 缺点 过于简单的项目不适用、大型的项目视图状态较多时构建和维护成本太大

合理的运用架构模式有利于项目、团队开发工作，但是到底选择哪个设计模式，哪种设计模式更好，就像本文开头所说，不同的设计模式，只是让不同的场景有了更多的选择方案。根据项目场景和开发需求，选择最合适的解决方案。

2.关于RAC你有怎样运用到解决不同API依赖关系

- 信号的依赖：使用场景是当信号A执行完才会执行信号B,和请求的依赖很类似,例如请求A请求完毕才执行请求B,我们需要注意信号A必须要执行发送完成信号,否则信号B无法执行



```
//这相当于网络请求中的依赖,必须先执行完信号A才会执行信号B
//经常用作一个请求执行完毕后,才会执行另一个请求
//注意信号A必须要执行发送完成信号,否则信号B无法执行
RACSignal * concatSignal = [self.signalA concat:self.signalB]
//这里我们是对这个拼接信号进行订阅
[concatSignal subscribeNext:^(id x) {
    NSLog(@"%@",x);
}
```

```
}];
```

3.@weakify和我们宏定义的weakSelf有什么区别？

- @weakify 可以多参数使用

4.微服务架构设想。

微服务架构具有以下优势：

- 1.灵活和独立的可扩展性

灵活扩展是微服务架构的主要优势之一。与单片架构不同，每个模块都可以水平扩展并独立于其他模块。因此，微服务架构非常适合大型项目。

- 2.独立技术堆栈

在微服务架构中，软件工程师有机会使用各种工具和技术构建APP。代码可以用不同的编程语言编写，这为APP开发过程增加了更多的灵活性。

- 3.更好的故障隔离

如果一个服务失败，它不会影响其他服务的功能。与其他体系结构样式相比，在微服务中，系统继续工作，单片模式下的问题会影响整个APP。

- 4.易于部署和集成

虽然即使是小代码更改的情况下，开发人员也必须再次部署APP，但在微服务架构中，部署变得更快更轻松。

由于所有服务都是围绕单一业务流程构建的，因此程序员不必修改和重新部署整个APP，只需要您需要的区域。因此，改进产品也比较简单。

微服务可以通过全自动部署机制独立部署。此外，通过使用开源持续集成工具，开发人员大大

简化了与第三方服务的集成。

- **5.容易理解**

微服务体系结构的另一个优点是很容易理解系统是如何工作的以及它是如何开发的。当一个新的团队成员来到这个项目并且必须快速钻研它时，这特别有用。

那么在iOS中如何借鉴这种思想去构建我们的App呢？是需要我们开发者自己去不断探索的

二，设计模式篇

1.iOS有哪些常见的设计模式？

- **单例模式：**

单例保证了应用程序的生命周期内仅有一个该类的实例对象,而且易于外界访问.在ios sdk中,UIApplication, NSBundle, NSNotificationCenter, NSFileManager, NSUserDefaults, NSURLCache等都是单例.

- **委托模式：**

委托Delegate是协议的一种,通过@protocol方式实现，常见的有tableView，textField等。

- **观察者模式：**

观察者模式定义了一种一对多的依赖关系，让多个观察者对象同时监听某一个主题对象。在iOS中,观察者模式的具体实现有两种: 通知机制(notification)和KVO机制(Key-value Observing)

2.单例会有什么弊端？

- **主要优点：**

1、提供了对唯一实例的受控访问。

2、由于在系统内存中只存在一个对象，因此可以节约系统资源，对于一些需要频繁创建和销毁的对象单例模式无疑可以提高系统的性能。

3、允许可变数目的实例。

- **主要缺点：**

1、由于单例模式中没有抽象层，因此单例类的扩展有很大的困难。

2、单例类的职责过重，在一定程度上违背了“单一职责原则”。

3、滥用单例将带来一些负面问题，如为了节省资源将数据库连接池对象设计为的单例类，可能会导致共享连接池对象的程序过多而出现连接池溢出；如果实例化的对象长时间不被利用，系统会认为是垃圾而被回收，这将导致对象状态的丢失。

3.编程中的六大设计原则是哪些？

- **1.单一职责原则**

通俗地讲就是一个类只做一件事

CALayer：动画和视图的显示。

UIView：只负责事件传递、事件响应。

- **2.开闭原则**

对修改关闭，对扩展开放。要考虑到后续的扩展性，而不是在原有的基础上来回修改

- **3.接口隔离原则**

使用多个专门的协议、而不是一个庞大臃肿的协议，如 UITableViewDelegate + UITableViewDataSource

- **4.依赖倒置原则**

抽象不应该依赖于具体实现、具体实现可以依赖于抽象。调用接口感觉不到内部是如何操作的

- **5.里氏替换原则**

父类可以被子类无缝替换，且原有的功能不受任何影响 如：KVO

- **6.迪米特法则**

一个对象应当对其他对象尽可能少的了解，实现高聚合、低耦合

4.如何设计一个图片缓存框架？

- 可以模仿 SDWebImage 来实现。
- 构成
- Manager
- 内存缓存
- 磁盘缓存
- 网络下载
- Code Manager

图片解码

图片解压缩

- 图片的存储是以图片的单向 hash 值为 Key

内存设计需要考虑的问题

存储的 Size

因为内存的空间有限，我们针对不同尺寸的图片，给出不同的方案

- 10K 以下的50个
- 100Kb 以下的20个
- 100kb 以上的10个

淘汰的策略

内存的淘汰策略 采取 LRU（最近最少使用算法）

触发淘汰策略的时机有三种

- 1.定期检查（不建议，耗性能）
- 2.提高检查触发频率（一定要注意开销）
- 1.前后台切换的时候
- 2.每次读写的时候

磁盘设计需要考虑的问题

存储方式

大小限制（有固定的大小）

移除策略（可以设置为7天或者15天）

- 网络设计需要考虑的问题

图片请求的最大并发量

请求超时策略

请求优先级

图片解码

- 应用 策略模式，针对 jpg、png、gif 等不同的图片格式进行解码
- 图片解码的时机

- 在子线程 图片刚下载完时
- 在子线程 刚从磁盘读取完时
- 避免在主线程解压缩、解码，避免卡顿

5.如何设计一个时长统计框架？

记录器

- 页面式记录器
- 流式记录器
- 自定义式

记录管理者

- 内存记录缓存
- 磁盘存储
- 上传器

如何降低数据的丢失率？

- 定期写入磁盘
- 每当达到某个值的时候，就写入磁盘

记录上传的时机

- 前后台切换的时候可以上传
- 从无网到有网切换的时候可以上传

上传时机的选择

- 立即上传
- 定时上传

- 延时上传

部分总结的架构+设计模式面试题就到这里了，
以上内容对你有帮助的话，记得关注我们！

文章来源于网络，已尽可能标明作者以及来源，文章内容为作者独立观点，不代表本公众号立场，因文章侵权本公众号不承担任何法律及连带责任，如有侵权，请联系我们删除。



觉得不错的话，别忘了点个"在看"哦~ 📌

收录于话题 #BAT大厂面试题合集·9个

上一篇

[iOS面试题合集（组件化+性能优化篇）](#)

下一篇

[iOS面试题合集（RunLoop篇）](#)

[阅读原文](#)

喜欢此内容的人还喜欢

审查滴滴，释放了一个严厉的信号

21世纪商业评论

细思极恐！厦门网友爆料：搬家后房东清楚地说出我的新住址，还说监控随便查，什么情况？

小鱼网

“老婆，对不起，我走了”凌晨突发！厦门一男子跨越同安大桥栏杆，欲轻生...

小鱼网